

Service Layers Explained

Coordinating Business Logic in Apex

FFLib Series · Session 002

Andrew Fawcett

Code With Sally

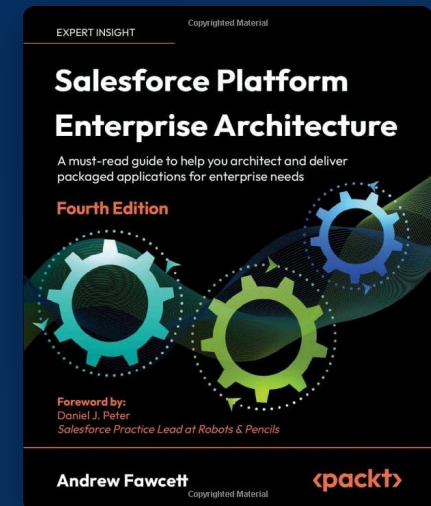


About me

Independent Salesforce Consultant | Former CPO,
Salesforce, Heroku | CTO, FinancialForce.com

- **Independent consultant and architect** — Salesforce platform development, PaaS integrations, and agentic AI
- **Open source and guidance** — creator of DLRS, Apex frameworks, and *Salesforce Platform Enterprise Architecture*
- **Salesforce Customers and Partners Advisory** — architecture and scale, AI-enhanced workflows, org health and codebase mentoring, and ISV product advisory

Reduce complexity, unblock engineering, and leave architectures — and teams — able to evolve.



The series

SESSION



Session #1 - Separation of Concerns in Apex: Why Your Future Self Will Thank You



Session #2 - Service Layers Explained: Coordinating Business Logic in Apex



Session #3 - Domain vs Service: Where Should Your Apex Logic Live?



Session #4 - Query Logic as a First-Class Architecture Concern

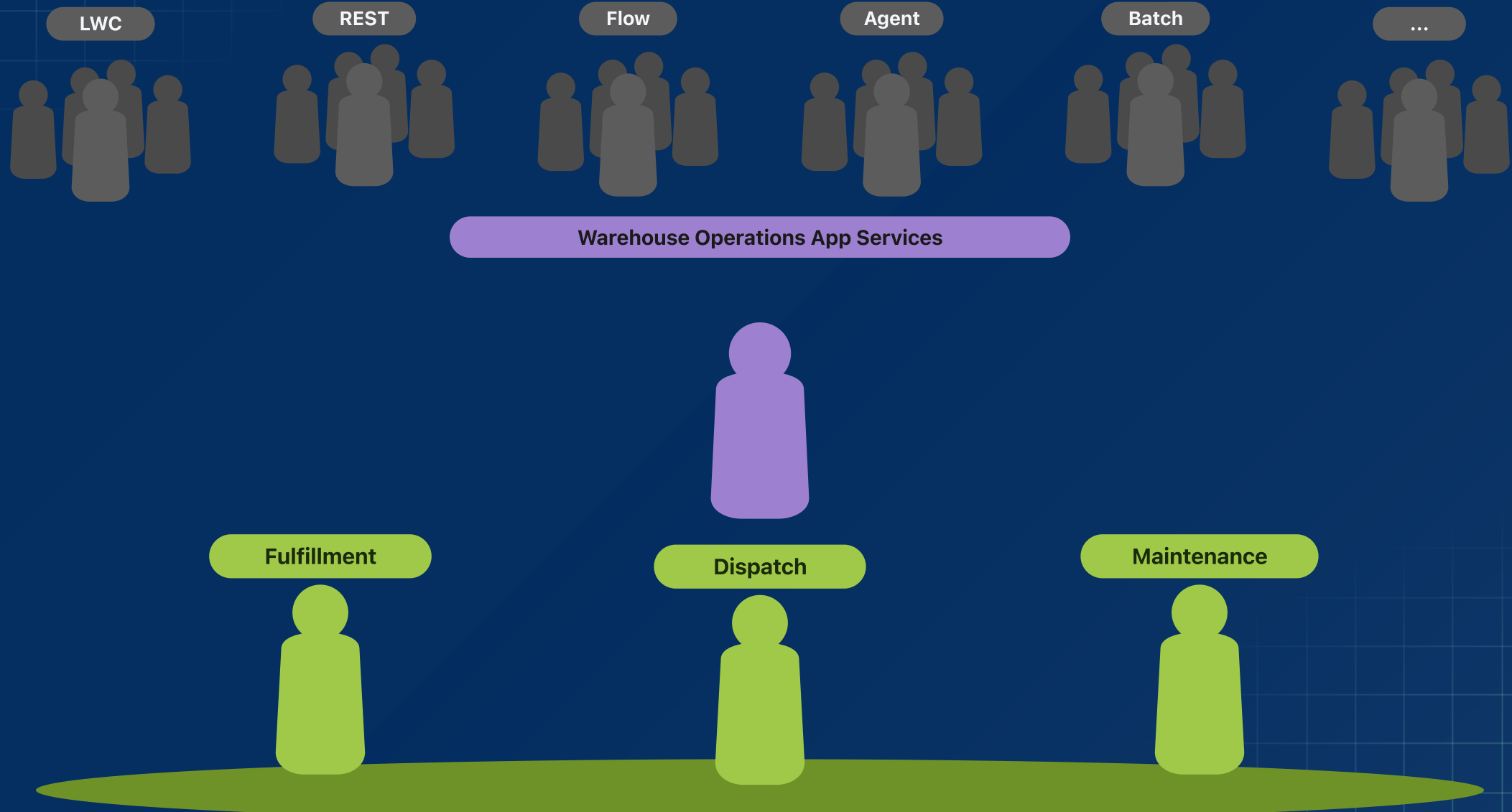


Session #5 - Mocking in Apex: Why It Changes Everything



Session #6 - Enterprise-Scale Apex Across Multiple Packages

Coordinating Business Logic in Apex



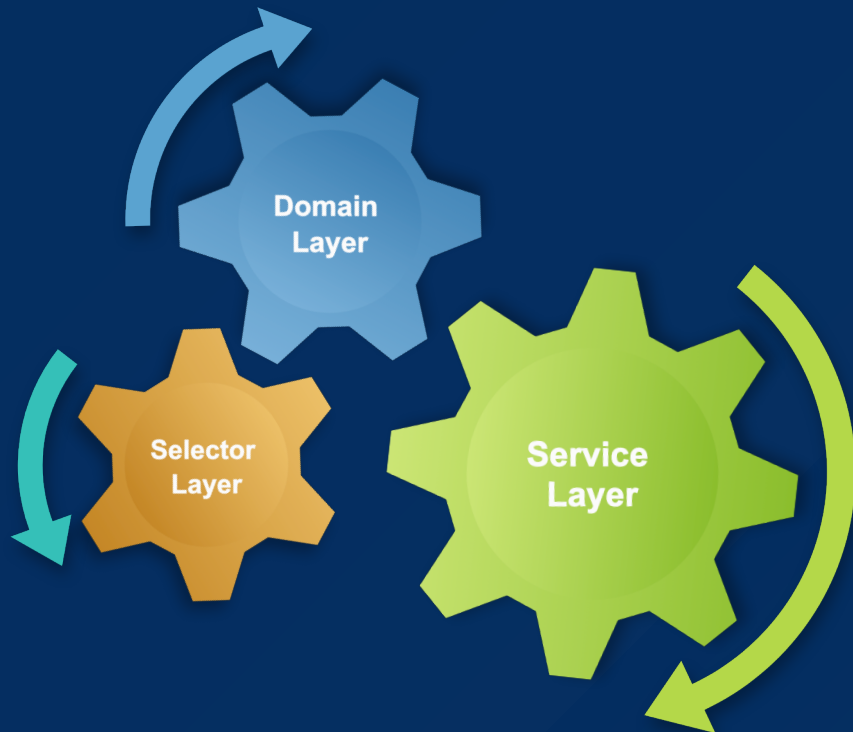
Service Layer

“ *Defines an application's boundary with a layer of services that establishes a set of available operations and coordinates the application's response in each operation.* ”

[Martin Fowler](#) — *Patterns of Enterprise Application Architecture*



Service Layers Explained



SERVICE LAYERS EXPLAINED

Separation of Concerns Recap ☐

The Unit of Work ☐

Service Layer Principles ☐

Warehouse Operations App Overview ☐

Warehouse Operations App Code ☐

Warehouse Operations Agent ☐

Salesforce Platform Layers

Presentation

- **You define:** Layouts, Custom Buttons, Flow, Record Types, Formula, Reports, Dashboards, Agent Builder
- **You code:** Apex Controllers, Visualforce, Aura, Lightning Components, React

Integration

- **You define:** Custom Objects, Custom Fields
- **You code:** Apex SOAP, Apex REST, Invocable Methods, Agent Actions, MCP, `global` Apex

Business Logic

- **You define:** Formula, Validation Rules, Workflow, Process Builder, Flow, Sharing Rules, Prompt Builder
- **You code:** Apex Services, DataWeave, Callouts

Data Access

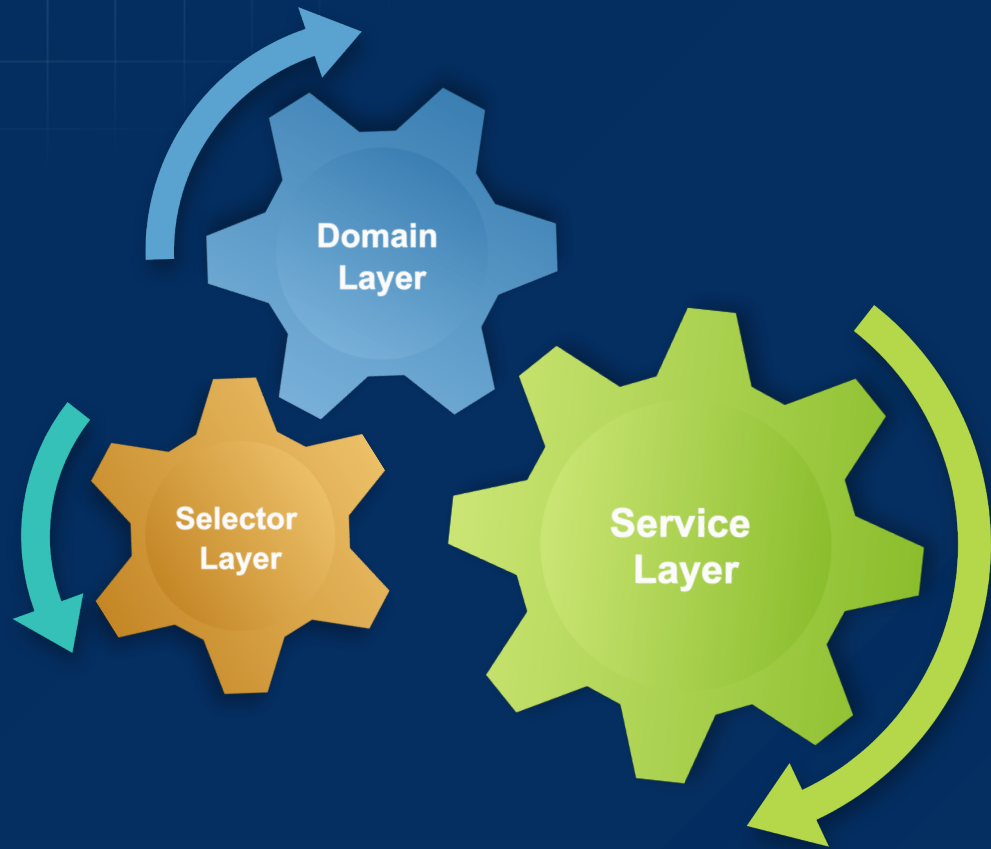
- **You define:** Data Loaders, Permission Sets, Profiles
- **You code:** SOQL, SOSL and Salesforce APIs

Database

- **You define:** Custom Objects, Custom Fields, Relationships, Rollups, D360
- **You code:** Apex Triggers, External Data Sources (External Objects)



Apex Enterprise Patterns (fflib) - Color Chart



-  Client
-  Service
-  Domain
-  Trigger Handler
-  Selector

Suggested Class Folder Layout

Presentation

■ FulfillmentReleaseController.cls ■ WarehouseDispatchController.cls ■ MaintenanceCompleteController.cls .

// /classes/controllers

Integration

■ ReleaseOrders.cls ■ DispatchWarehouse.cls ■ FulfillmentResource.cls

// /classes/actions · /classes/restapis



Business Logic

■ FulfillmentService.cls ■ DispatchService.cls ■ MaintenanceService.cls

// /classes/services – named for the process

■ Robots.cls ■ FulfillmentOrders.cls ■ FulfillmentLines.cls ...

// /classes/domains – named for the object

Data Access

■ RobotsSelector.cls ■ FulfillmentLinesSelector.cls ■ WarehousesSelector.cls

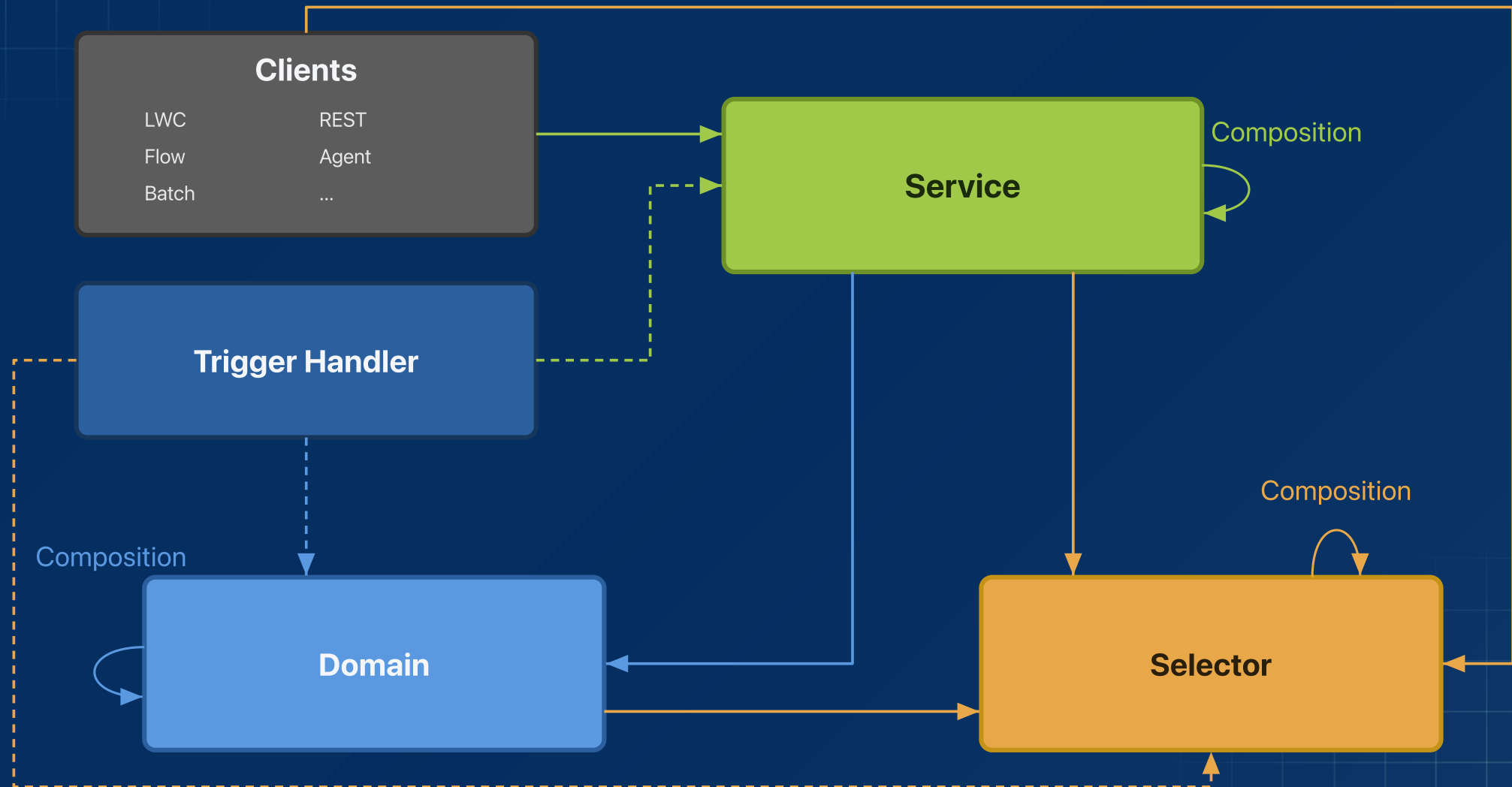
// /classes/selectors

Database

■ RobotsTriggerHandler.cls

// /classes/triggerHandlers

Who can call whom



Who can call whom

CALLER	Service	Domain	Selector
Client (LWC, REST, Flow, Agent, Batch, ...)	•		•
Trigger Handler	•	•	•
Service	•	•	•
Domain		•	•
Selector			•

How durable is your code when change occurs?

How users interact with your application changes a lot — protect against it with SoC



🤔 Business Logic Impact

WarehouseDispatchController.cls DispatchWarehouse.cls FulfillmentReleaseController.cls ReleaseOrders.cls

```
// logic copied — every new client type
VF · Aura · LWC · Agent Action · React · Next?
└─ WarehouseDispatchController.dispatchWarehouse(...)
└─ DispatchWarehouse.execute(...)
└─ FulfillmentReleaseController.releaseOrder(...)
└─ ReleaseOrders.execute(...)
```

① Claudeforce. Salesforce + Anthropic, August 2026 — yet another interaction layer on your application.

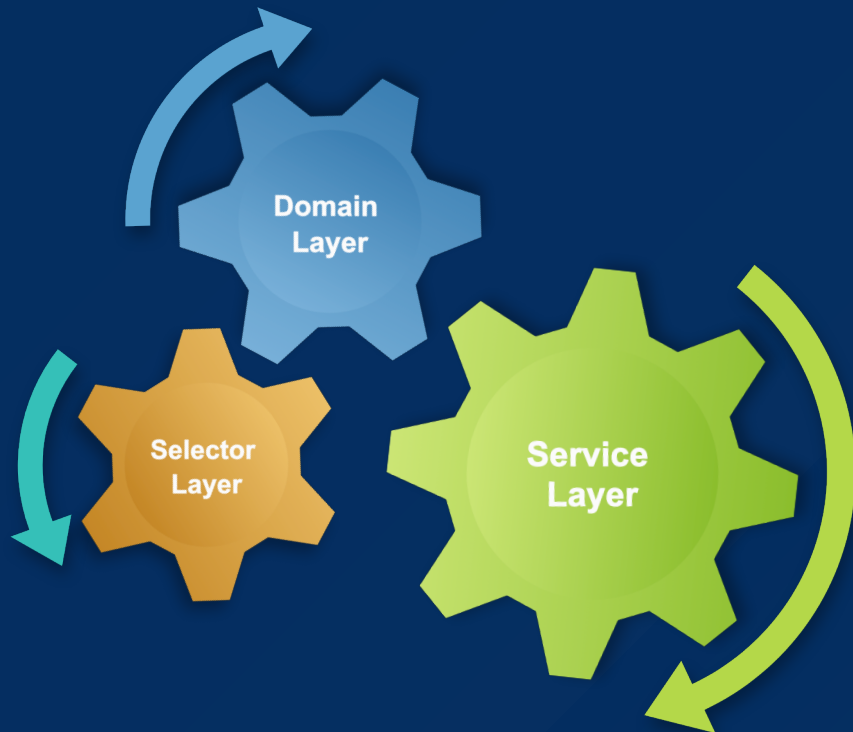


DispatchService.cls FulfillmentService.cls

```
// same services — every client type
LWC · REST · Batch · Flow · Agent Action · Next?
└─ DispatchService.dispatchWarehouses(...)
└─ FulfillmentService.releaseOrders(...)
```

❤️ Your business logic is **the** most important logic in your app — and when it's everywhere, it becomes a **significant investment risk**

Service Layers Explained



SERVICE LAYERS EXPLAINED

Separation of Concerns Recap



The Unit of Work



Service Layer Principles



Warehouse Operations App Overview



Warehouse Operations App Code



Warehouse Operations Agent



Unit of Work

- [Martin Fowler](#) — *Patterns of Enterprise Application Architecture*
 - “Maintains a list of objects affected by a business transaction and coordinates the writing out of changes and the resolution of concurrency problems.”
- One transaction: work completes or is rolled back explicitly
- Best practices: automatically bulkifies, runs in user mode
- Relationships: built in memory without having to insert parent
- `commitWork()` takes a savepoint and rolls back if anything fails
- In Apex that means: register work now, `commitWork()` once — see [AndyInTheCloud, June 2013](#)

Without Unit Of Work — Lines: 73

■ Opportunity setup — raw DML

```
1 List<Opportunity> opps = new List<Opportunity>();
2 List<List<Product2>> productsByOpp = new List<List<Product2>>();
3 List<List<PricebookEntry>> pbesByOpp = new List<List<PricebookEntry>>();
4 List<List<OpportunityLineItem>> linesByOpp = new List<List<OpportunityLineItem>>();
5 for (Integer o = 0; o < 10; o++) {
6     Opportunity opp = new Opportunity();
7     // ... set Name, StageName, CloseDate
8     opps.add(opp);
9     List<Product2> products = new List<Product2>();
10    List<PricebookEntry> pbes = new List<PricebookEntry>();
11    List<OpportunityLineItem> lines = new List<OpportunityLineItem>();
12    for (Integer i = 0; i < o + 1; i++) {
13        Product2 product = new Product2();
14        // ... set Name
15        products.add(product);
16        PricebookEntry pbe = new PricebookEntry();
17        // ... set UnitPrice, IsActive, Pricebook2Id
18        pbes.add(pbe);
19        OpportunityLineItem oli = new OpportunityLineItem();
20        // ... set Quantity, TotalPrice
21        lines.add(oli);
22    }
23    productsByOpp.add(products);
24    pbesByOpp.add(pbes);
25    linesByOpp.add(lines);
26 }
27 insert opps;
28 List<Product2> allProducts = new List<Product2>();
29 for (List<Product2> products : productsByOpp) {
30     allProducts.addAll(products);
31 }
32 insert allProducts;
33 Integer oppIdx = 0;
34 List<PricebookEntry> allPbes = new List<PricebookEntry>();
35 for (List<PricebookEntry> pbes : pbesByOpp) {
36     List<Product2> products = productsByOpp[oppIdx++];
37     Integer lineIdx = 0;
38     for (PricebookEntry pbe : pbes) {
39         pbe.Product2Id = products[lineIdx++].Id;
40     }
41     allPbes.addAll(pbes);
42 }
43 insert allPbes;
44 oppIdx = 0;
45 List<OpportunityLineItem> allLines = new List<OpportunityLineItem>();
46 for (List<OpportunityLineItem> lines : linesByOpp) {
47     List<PricebookEntry> pbes = pbesByOpp[oppIdx];
48     Integer lineIdx = 0;
49     for (OpportunityLineItem oli : lines) {
50         oli.OpportunityId = opps[oppIdx].Id;
51         oli.PricebookEntryId = pbes[lineIdx++].Id;
52     }
53     allLines.addAll(lines);
54     oppIdx++;
55 }
56 insert allLines;
```

■ Zoom · lines 1–4

```
1 List<Opportunity> opps = new List<Opportunity>();
2 List<List<Product2>> productsByOpp = new List<List<Product2>>();
3 List<List<PricebookEntry>> pbesByOpp = new List<List<PricebookEntry>>();
4 List<List<OpportunityLineItem>> linesByOpp = new List<List<OpportunityLineItem>>();
```

■ Zoom · lines 47–56

```
47 for (List<PricebookEntry> pbes : pbesByOpp) {
48     List<Product2> products = productsByOpp[oppIdx++];
49     Integer lineIdx = 0;
50     for (PricebookEntry pbe : pbes) {
51         pbe.Product2Id = products[lineIdx++].Id;
52     }
53     allPbes.addAll(pbes);
54 }
55 }
56 }
```

■ Zoom · lines 36, 43, 57, 73

```
36 insert opps;
43 insert allProducts;
57 insert allPbes;
73 insert allLines;
```

■ Zoom · Transaction control

```
1 Savepoint sp = Database.setSavepoint();
2 try {
3     // ... lists, stitch, four inserts
4 } catch (Exception e) {
5     Database.rollback(sp);
6     throw new DispatchServiceException(e.getMessage(), e);
7 }
```

With Unit of Work — Lines: 31 (58% ↓)

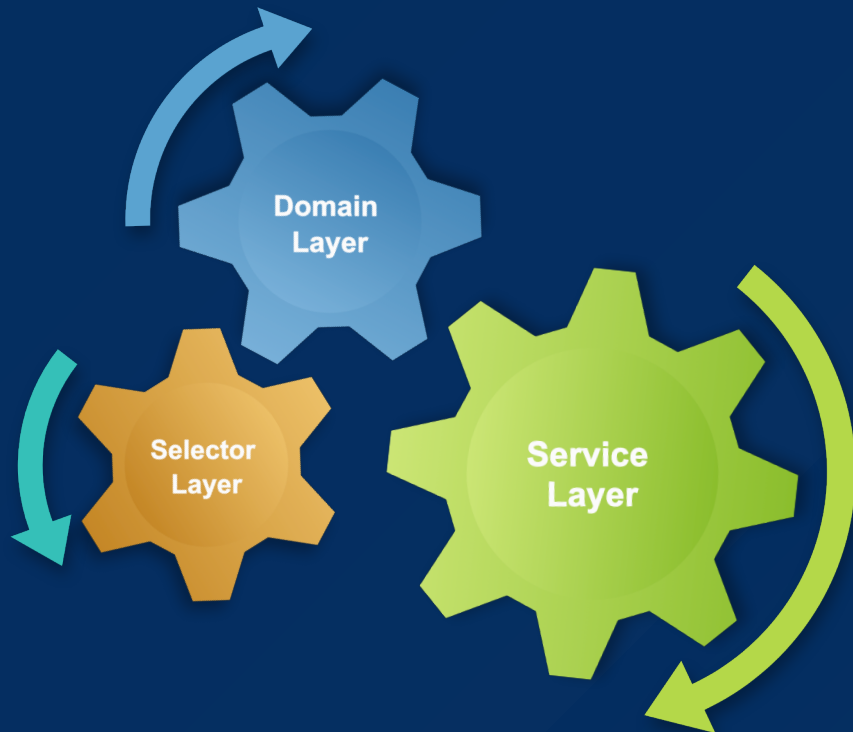
■ Same work — register, then commit

```
1 fflib_SObjectUnitOfWork uow = new fflib_SObjectUnitOfWork(  
2     new List<Schema.SObjectType> {  
3         Product2.SObjectType,  
4         PricebookEntry.SObjectType,  
5         Opportunity.SObjectType,  
6         OpportunityLineItem.SObjectType  
7     });  
8 for (Integer o = 0; o < 10; o++) {  
9     Opportunity opp = new Opportunity();  
10    // ... set Name, StageName, CloseDate  
13    uow.registerNew(opp);  
14    for (Integer i = 0; i < o + 1; i++) {  
15        Product2 product = new Product2();  
16        product.Name = opp.Name + ' : Product : ' + i;  
17        uow.registerNew(product);  
18        PricebookEntry pbe = new PricebookEntry();  
19        // ... set UnitPrice, IsActive, Pricebook2Id  
23        uow.registerNew(pbe, PricebookEntry.Product2Id, product);  
24        OpportunityLineItem oli = new OpportunityLineItem();  
25        // ... set Quantity, TotalPrice  
27        uow.registerRelationship(oli, OpportunityLineItem.PricebookEntryId, pbe);  
28        uow.registerNew(oli, OpportunityLineItem.OpportunityId, opp);  
29    }  
30 }  
31 uow.commitWork();
```

fflib_SObjectUnitOfWork Methods

- `fflib_SObjectUnitOfWork(List<SObjectType> types)` — dependency order
- `registerNew(record)` · `registerNew(record, field, parent)`
- `registerRelationship(record, field, related)` — stitch before Ids exist
- `registerDirty(record)` · `registerDeleted(record)`
- `commitWork()` — savepoint, bulk DML, rollback, rethrow
- Other methods: `registerWork` , `registerUpsert` , `registerEmail` ...

Service Layers Explained



SERVICE LAYERS EXPLAINED

Separation of Concerns Recap



The Unit of Work



Service Layer Principles



Warehouse Operations App Overview



Warehouse Operations App Code



Warehouse Operations Agent

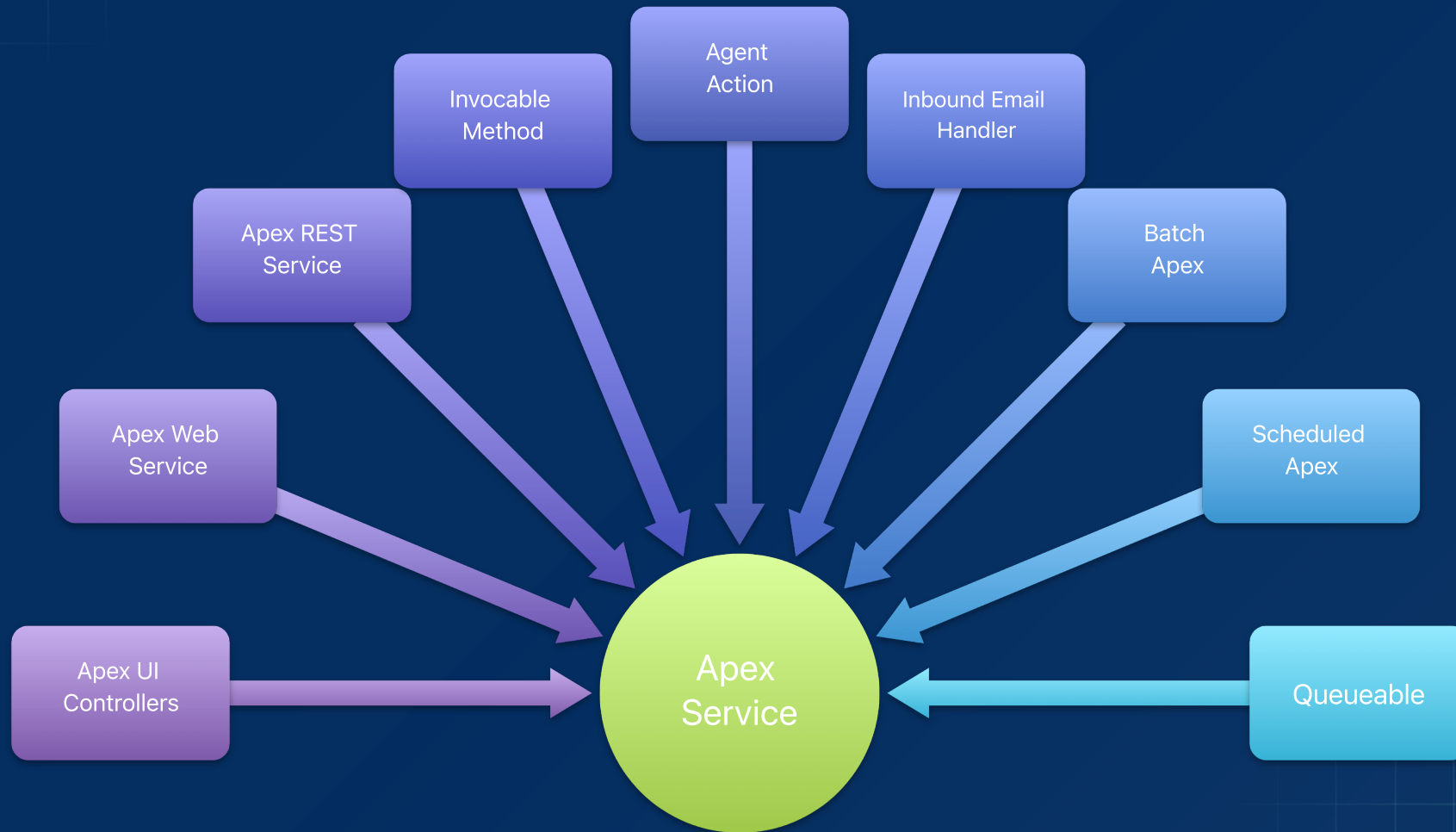


Service ➔ Task Orientated Logic

- A Service represents a **feature** of the application
 - Fulfillment — `FulfillmentService.cls`
 - Dispatch — `DispatchService.cls`
 - Maintenance — `MaintenanceService.cls`
- Each method is a **task** within that feature — `releaseOrders`, `dispatchWarehouses`, `completeLines`
- Think of it as the **conductor** — the band handles querying and object-specific logic; the Service sets the tempo and the rules of the orchestration



Services have many Consumers



Service Checklist

- Client-agnostic inputs and outputs
 -  `dispatchWarehouses(Set<Id> warehouseIds)`
 -  `dispatch(DispatchForm form)`
- Client-agnostic exceptions
 -  `throw DispatchServiceException`
 -  `throw AuraHandledException`
- **Bulkification** — collections in and out; don't force callers to loop
 -  `dispatchWarehouses(Set<Id> warehouseIds)`
 -  `dispatchWarehouse(Id warehouseId)` only
- **Transactions** — one UOW; rollback; Service→Service passes the outer uow
- **Security** — enforce user-mode in DML (UOW) and SOQL (Selector)

Service Evolution

- **Services** — now prefer instance methods over static
 -  `DispatchService.newInstance().dispatchWarehouses(...)` — reduces boilerplate to support mocking, better alignment with Apex Stubs requirements
 -  `public static void dispatchWarehouses(...)`
- **Apex interfaces** — now recommended only for dependency injection
 -  `IDispatchService` use when decoupling implementation e.g. packaging *
 -  `IFulfillmentService` / `IMaintenanceService` on every service by default
- **Application class** — optional; mocking can be done without it — [blog](#)
 - Demonstrated in this sample code via constructor and property injection

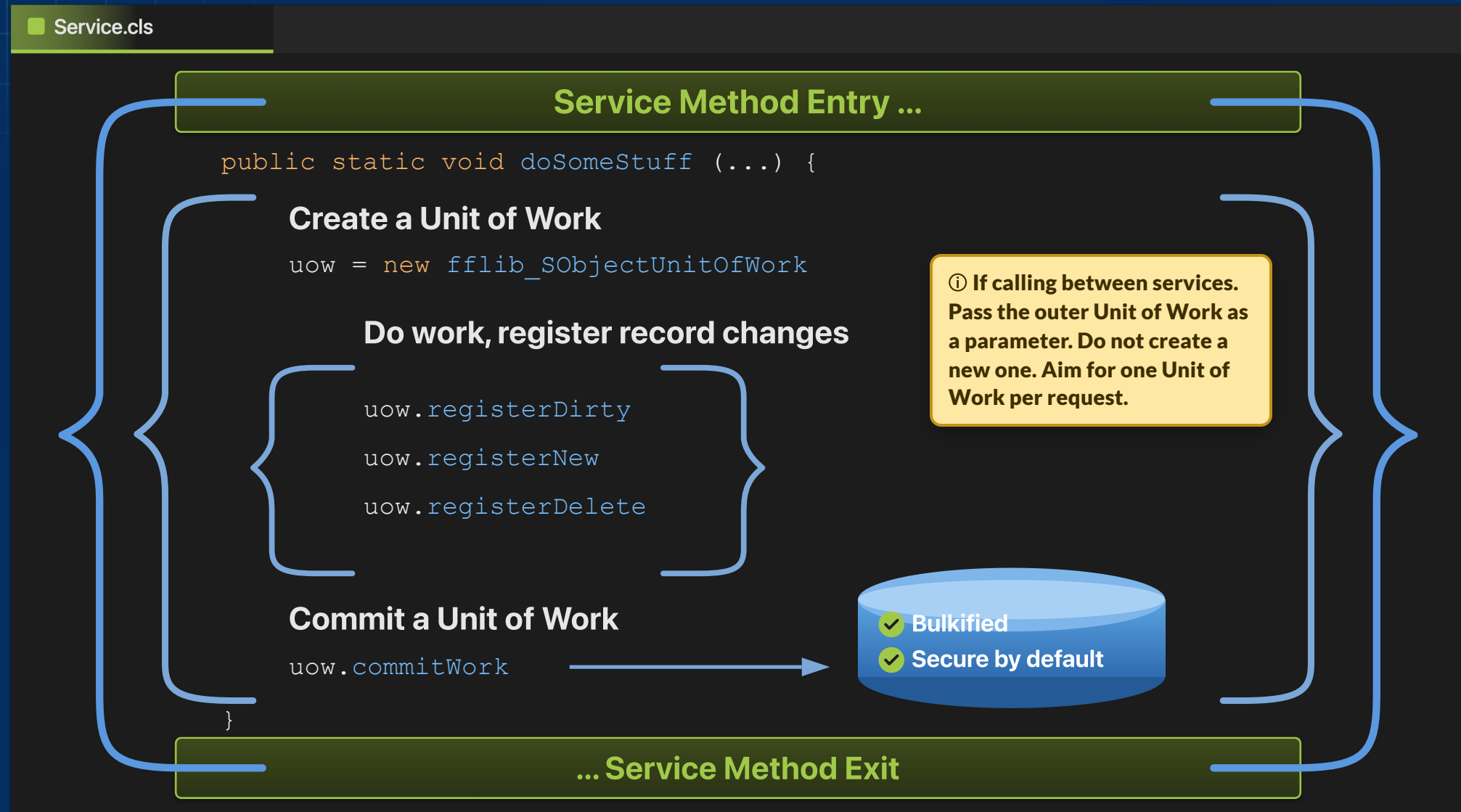
① * **Session #6 — Enterprise-Scale Apex Across Multiple Packages.**

Metadata driven Dependency Injection (DI) and Apex Interfaces for resolving Services implementations will be covered in this session.

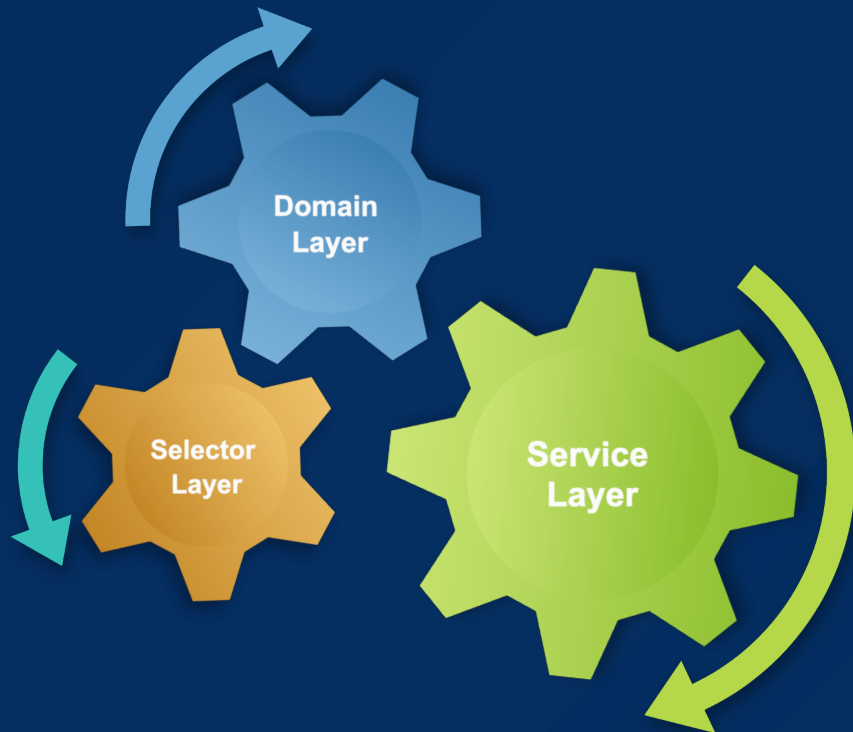
Service - Consumers and Dependencies



Service - Leveraging Unit of Work



Service Layers Explained



SERVICE LAYERS EXPLAINED

Separation of Concerns Recap



The Unit of Work



Service Layer Principles



Warehouse Operations App Overview



Warehouse Operations App Code



Warehouse Operations Agent



Warehouse Operations App

- The operations app for a warehouse that uses robots
- Release pick work, dispatch idle robots, take a worn one into maintenance
- The sample is the **desk**, not the robot brain
- Improve sample clarity between service vs domain
 - `DispatchService` — a process
 - `Robots` — an object
- Leverages multiple clients, LWC, REST API, Batch and AI



Warehouse App Tabs

Warehouse Operations

HomeWarehouses ▾Robots ▾Robot Models ▾Fulfillment Orders

Warehouse

North Hub

Related

Details

Warehouse Name

North Hub

Status

Active

Location

North

Owner

User User

System Information

Created By

User User, 9/6/2026, 5:29 AM

Last Modified By

User User, 9/6/2026, 5:29 AM

Warehouse Operations

HomeWarehouses ▾Robots ▾Robot Models ▾Fulfillment Orders

Robot

Ember

Related

Details

Robot Name

Ember

Warehouse

North Hub

Model

Picker 100

Status

Idle

Health

Warning

Wear Percent

79.00%

Hours Since Service

39.00

Battery Level

55.00

Owner

Warehouse Operations

HomeWarehouses ▾Robots ▾Robot Models ▾Fulfillment Orders

Fulfillment Order

FO-00000

Related

Details

Fulfillment Order Number

FO-00000

Warehouse

North Hub

Status

Draft

Priority

Normal

Due Date

9/9/2026

Owner

User User

System Information

Warehouse Operations

HomeWarehouses ▾Robots ▾Robot Models ▾Fulfillment Orders

Fulfillment Order

FO-00001

Related

Details

Fulfillment Lines (3)

New

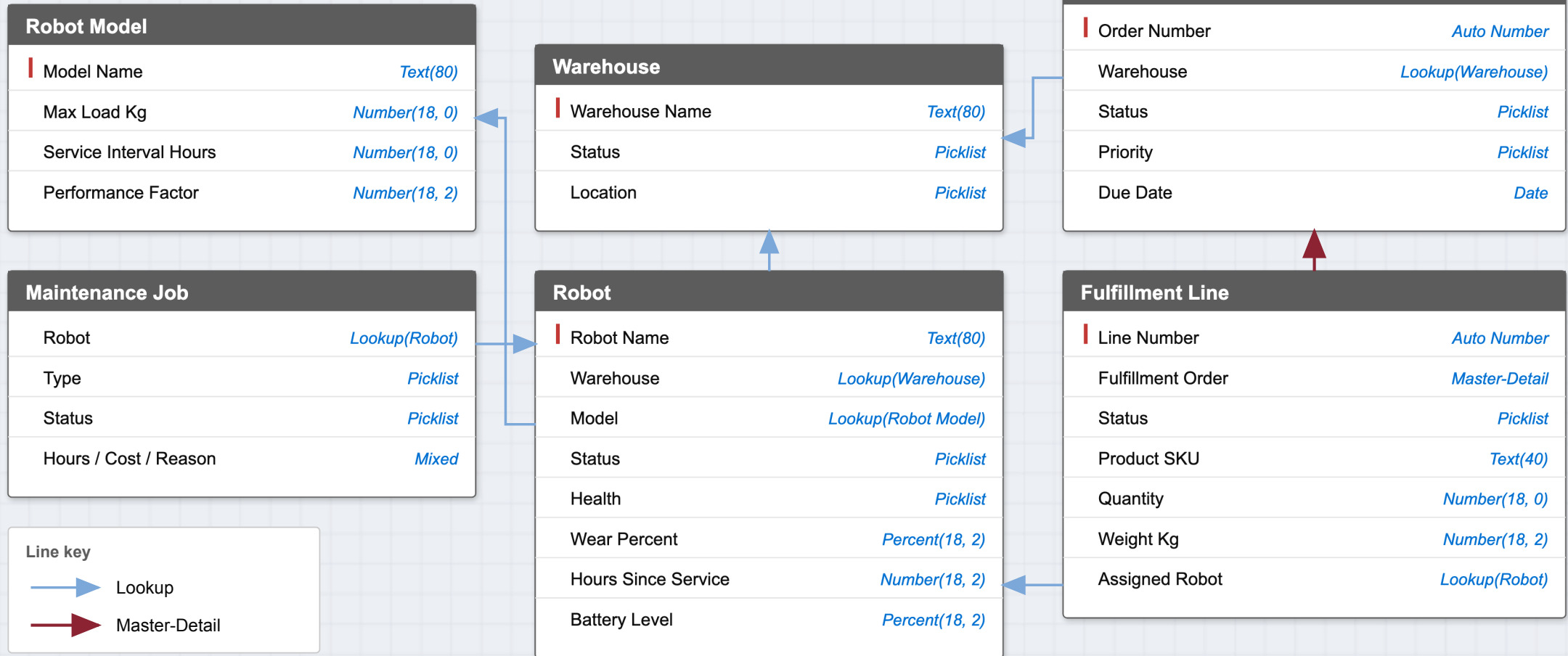
Fulfillment Line Number

FL-00001

FL-00002

FL-00003

Warehouse App Objects

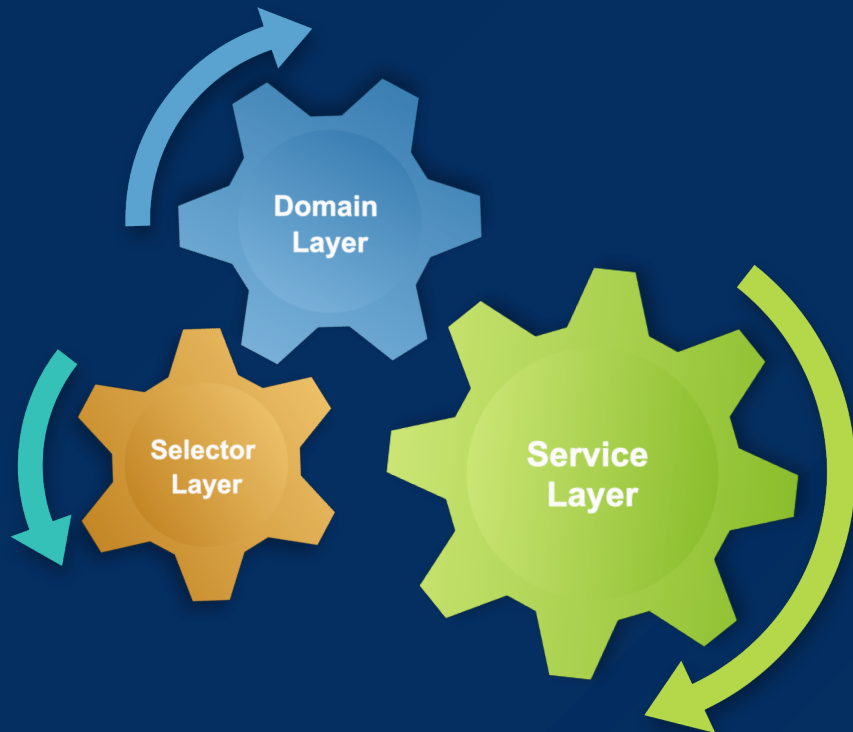


Warehouse App Processes

North Hub Warehouse Fulfillment



Service Layers Explained



SERVICE LAYERS EXPLAINED

Separation of Concerns Recap



The Unit of Work



Service Layer Principles



Warehouse Operations App Overview



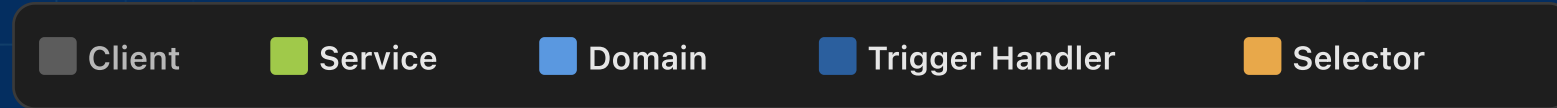
Warehouse Operations App Code



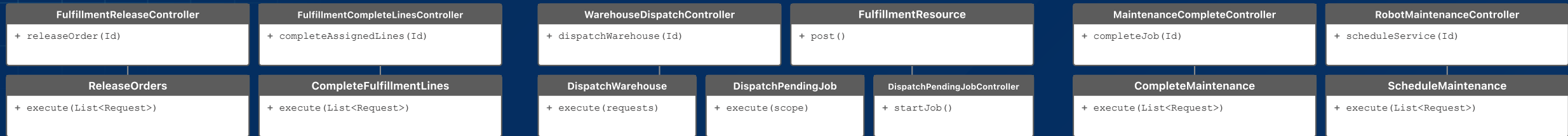
Warehouse Operations Agent



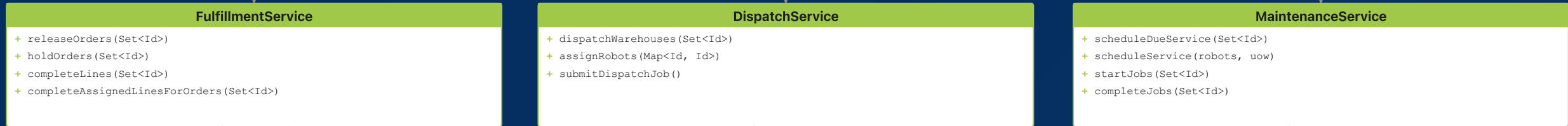
Warehouse App Classes



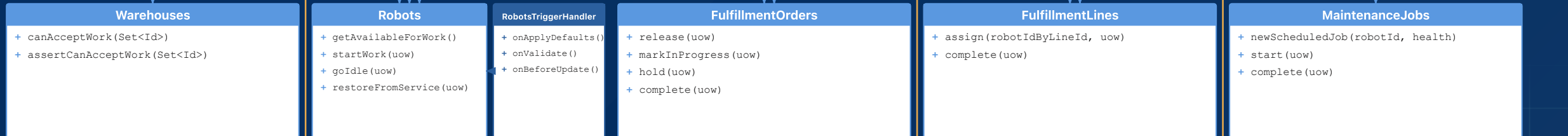
Clients



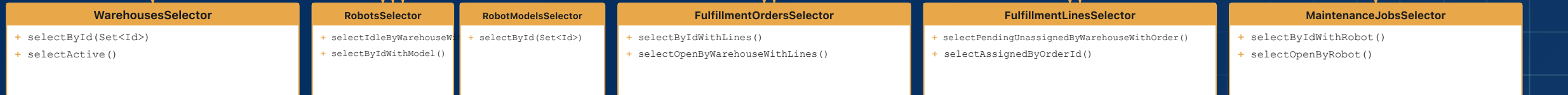
Services



Domains



Selectors



North Hub Warehouse Fulfillment

1 Release FO-00001

Put work on the floor

FulfillmentReleaseController.cls

@AuraEnabled
releaseOrder (Id)

FulfillmentService.cls

releaseOrders (Set<Id>)

FulfillmentOrders.cls

release (uow)

FulfillmentOrdersSelector.cls

selectByIdWithLines (Set<Id>)

2 Dispatch North Hub

Put robots on that work

WarehouseDispatchController.cls

@AuraEnabled
dispatchWarehouse (Id)

DispatchService.cls

dispatchWarehouses (Set<Id>)

Robots.cls

getAvailableForWork ()

RobotsSelector.cls

selectIdleByWarehouseWithModel (Set<Id>)

3 Complete Lines FO-00001

Finish the picks

FulfillmentCompleteLinesController.cls

@AuraEnabled
completeAssignedLines (Id)

FulfillmentService.cls

completeAssignedLinesForOrders (Set<Id>)

MaintenanceService.cls

scheduleService (robots, uow)

Robots.cls

goIdle (uow) applyWear (uow)

4 Complete Maintenance The new job

Put Ember back

MaintenanceCompleteController.cls

@AuraEnabled
completeJob (Id)

MaintenanceService.cls

completeJobs (Set<Id>)

MaintenanceJobs.cls

complete (uow)

Robots.cls

restoreFromService (uow)



WarehouseDispatchController.cls

```
1 public inherited sharing class WarehouseDispatchController {
2
3     private DispatchService dispatchSvc = /* constructor initialized */;
4
5     @AuraEnabled(cacheable=false)
6     public static void dispatchWarehouse(Id warehouseId) {
7         new WarehouseDispatchController().handleDispatchWarehouse(warehouseId);
8     }
9
10    public void handleDispatchWarehouse(Id warehouseId) {
11        try {
12            dispatchSvc.dispatchWarehouses(new Set<Id>{ warehouseId });
13        } catch (Exception e) {
14            throw toAura(e);
15        }
16    }
17
18    private static AuraHandledException toAura(Exception e) {
19        AuraHandledException auraException = new AuraHandledException(e.getMessage());
20        auraException.setMessage(e.getMessage());
21        return auraException;
22    }
```

DispatchPendingJob.cls

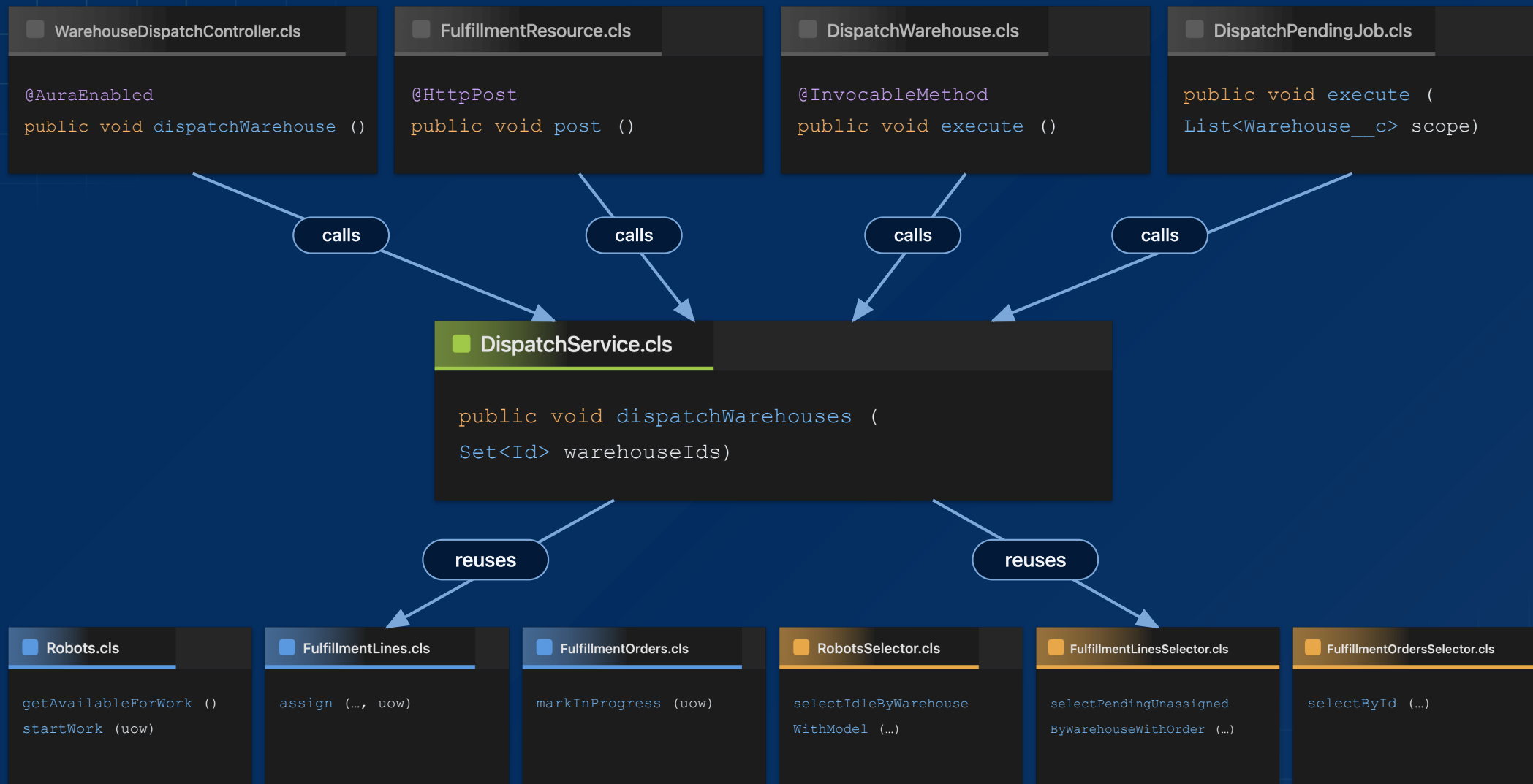
```
1 public inherited sharing class DispatchPendingJob
2     implements System.Schedulable, Database.Batchable<SObject>, Database.Stat
3
4     private WarehousesSelector warehouseSelector = /* constructor initialized */
5     private DispatchService dispatchSvc = /* constructor initialized */;
6
7     public Database.QueryLocator start(Database.BatchableContext context) {
8         return warehouseSelector.selectActiveAsQueryLocator();
9     }
10
11     public void execute(Database.BatchableContext context, List<Warehouse__c> v
12         try {
13             dispatchSvc
14                 .dispatchWarehouses(new Map<Id, Warehouse__c>(warehouses).keySet())
15         } catch (Exception e) {
16             jobErrors.add(/* JobError */);
17         }
18     }
19
20     public void finish(Database.BatchableContext context) { /* email JobErrors
21 }
```

■ FulfillmentResource.cls

```
1 @HttpPost
● 2 global static void post() {
3     new FulfillmentResource().handlePost();
4 }
5
● 6 public void handlePost() {
7     Request body = (Request) JSON.deserialize(
8         RestContext.request.requestBody.toString(), Request.class);
9     String action = /* last URI segment */;
10    switch on action {
11        when 'release' {
● 12            fulfillmentSvc
● 13                .releaseOrders(new Set<Id>(body.orderIds));
14        }
15        when 'dispatch' {
● 16            dispatchSvc
● 17                .dispatchWarehouses(new Set<Id>(body.warehouseIds));
18        }
19        when 'complete' {
● 20            fulfillmentSvc
● 21                .completeLines(new Set<Id>(body.lineIds));
22    }
```

DispatchWarehouse.cls

```
1 @InvocableMethod(  
2     label='Dispatch Warehouse'  
3     category='Warehouse Operations')  
● 4 public static List<Result> execute(List<Request> requests) {  
5     return new DispatchWarehouse().handleExecute(requests);  
6 }  
7  
● 8 public List<Result> handleExecute(List<Request> requests) {  
9     Set<Id> warehouseIds = new Set<Id>();  
10    for (Request request : requests) {  
11        warehouseIds.add(/* resolve request.warehouseName */);  
12    }  
● 13    dispatchSvc.dispatchWarehouses(warehouseIds);  
14    List<Result> results = new List<Result>();  
15    for (Request request : requests) {  
16        Result result = new Result();  
17        result.warehouseName = request.warehouseName;  
18        result.warehouseId = /* resolved Id */;  
19        results.add(result);  
20    }  
21    return results;  
22 }
```



```
public virtual void dispatchWarehouses(Set<Id> warehouseIds) {
```

1. **Service** — `UnitOfWork.newInstance()`
2. **Selector** — pending unassigned lines at the warehouses
3. **Selector** — idle robots at the warehouses
4. **Domain** — `Robots.getAvailableForWork()` (not Critical, wear < 90, batt > 20)
5. **Service** — `match` (per warehouse, line weight vs robot max load)
6. **Domain** — `Robots.startWork(uow)` >
`FulfillmentLines.assign(robotIdByLineId, uow)` >
`FulfillmentOrders.markInProgress(uow)`
7. **Service** — `uow.commitWork()`

```
}
```

■ DispatchService.dispatchWarehouses

```
1 public virtual inherited sharing class DispatchService {
2
3     private FulfillmentLinesSelector lineSelector = /* constructor initialized
4     private RobotsSelector robotSelector = /* constructor initialized */;
5     private FulfillmentOrdersSelector orderSelector = /* constructor initialize
6
7     public virtual void dispatchWarehouses(Set<Id> warehouseIds) {
8         fflib_SObjectUnitOfWork uow = UnitOfWork.newInstance();
9
10        List<FulfillmentLine__c> pending =
11            lineSelector.selectPendingUnassignedByWarehouseWithOrder(warehouseIds
12        List<Robot__c> idle =
13            robotSelector.selectIdleByWarehouseWithModel(warehouseIds);
14
15        List<Robot__c> available = Robots.newInstance(idle).getAvailableForWork()
16        Map<Id, Id> robotIdByLineId = match(pending, available);
17        Robots.newInstance(robotsToStart).startWork(uow);
18        FulfillmentLines.newInstance(linesToAssign).assign(robotIdByLineId, uow);
19        FulfillmentOrders.newInstance(released).markInProgress(uow);
20
21        uow.commitWork();
22    }
23 }
```

```
1 @TestVisible
2 private static fflib_SObjectUnitOfWork mock;
3
4 public static fflib_SObjectUnitOfWork newInstance() {
5     if (mock != null) {
6         return mock;
7     }
8     return new fflib_SObjectUnitOfWork(
9         new List<SObjectType>{
10             Warehouse__c.SObjectType,
11             RobotModel__c.SObjectType,
12             Robot__c.SObjectType,
13             FulfillmentOrder__c.SObjectType,
14             FulfillmentLine__c.SObjectType,
15             MaintenanceJob__c.SObjectType
16         },
17         new fflib_SObjectUnitOfWork.UserModeDML()
18     );
19 }
```

```
public virtual void completeLines (Set<Id> lineIds) {
```

1. **Service** — `UnitOfWork.newInstance()`

2. **Selector** — lines by Id (`lineIds`)

3. **Domain** — `FulfillmentLines.complete(uow)` · hours by assigned robot

4. **Selector** — robots with model (`hoursByRobotId`)

5. **Domain** — `Robots.goIdle(uow)` · `applyWear(uow)`

6. **Selector** — parent orders with every sibling line

7. **Domain** — `FulfillmentOrders.complete(uow, lineIds)` · `Robots.getDueForService()`

8. **Service** — `MaintenanceService.scheduleService(dueRobots, uow)`

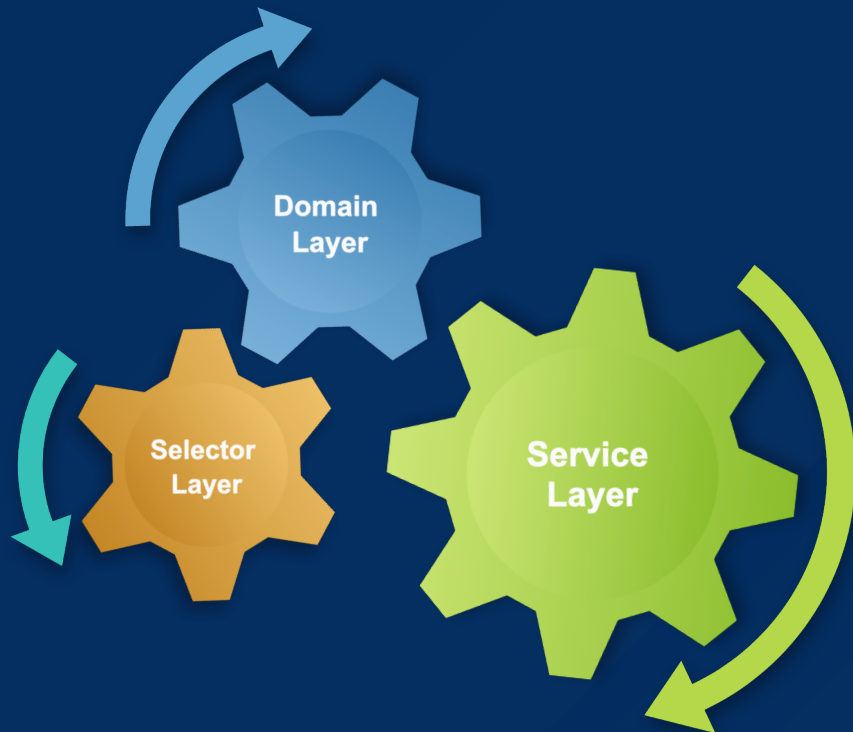
9. **Service** — `uow.commitWork()`

```
}
```

■ FulfillmentService.completeLines

```
1 public virtual void completeLines(List<FulfillmentLine__c> lines) {  
● 2     fflib_SObjectUnitOfWork uow = UnitOfWork.newInstance();  
3  
● 4     FulfillmentLines fulfillmentLines = FulfillmentLines.newInstance(lines);  
● 5     fulfillmentLines.complete(uow);  
● 6     Map<Id, Decimal> hoursByRobotId =  
7         fulfillmentLines.getEstimatedHoursByAssignedRobotId();  
● 8     List<Robot__c> robotRecords =  
9         robotSelector.selectByIdWithModel(hoursByRobotId.keySet());  
● 10    Robots robots = Robots.newInstance(robotRecords);  
● 11    robots.goIdle(uow); robots.applyWear(hoursByRobotId, uow);  
12  
● 13    List<FulfillmentOrder__c> orders = orderSelector.selectByIdWithLines(orderId);  
● 14    FulfillmentOrders.newInstance(orders).complete(uow, lineIds);  
15  
● 16    Set<Robot__c> dueRobots = robots.getDueForService();  
17    if (!dueRobots.isEmpty()) {  
● 18        maintenanceSvc.scheduleService(dueRobots, uow); // same uow  
19    }  
20  
● 21    uow.commitWork();  
22 }
```

Service Layers Explained



SERVICE LAYERS EXPLAINED

Separation of Concerns Recap



The Unit of Work



Service Layer Principles



Warehouse Operations App Overview



Warehouse Operations App Code



Warehouse Operations Agent



Warehouse Agent Actions



Warehouse Fulfillment

Actions

■ GetWarehouseDesk.cls

■ CreateRobot (Flow)

■ ReleaseOrders.cls

■ DispatchWarehouse.cls

■ CompleteFulfillmentLines.cls

■ CompleteMaintenance.cls

Services

→ ■ WarehouseService.cls

→ ■ FulfillmentService.cls

→ ■ DispatchService.cls

→ ■ FulfillmentService.cls

→ ■ MaintenanceService.cls

Warehouse Agent



Search...



Warehouse Operations

HomeWarehousesRobotsRobot ModelsMore

Robots

 All 

NewImport

3 items • Sorted by Robot Name • Updated a few seconds ago

☐	Robot Name ↑	Warehouse	Status	He
1	☐ Atlas	North Hub	Idle	He
2	☐ Bolt	North Hub	Idle	Wa
3	☐ Ember	North Hub	Idle	Wa

Warehouse Fulfillment



Let's chat!

I can review warehouse robots against pending fulfillment work, create robots when you are short, release draft orders, and run one dispatch. What would you like to do?

Describe your task or ask a question...

Warehouse Agent deploying Robots!





**Warehouse Operations**

[Home](#) [Warehouses](#) [Robots](#) [Robot Models](#) [More](#)

Robots

 **All** 

New

Import

11 items • Sorted by Robot Name • Updated a few seconds ago

	☐ Robot Name ↑	Warehouse	Status
1	☐ Atlas	North Hub	Working
2	☐ Bolt	North Hub	Working
3	☐ Ember	North Hub	Working
4	☐ North Spare 1	North Hub	Working
5	☐ North Spare 2	North Hub	Working
6	☐ North Spare 3	North Hub	Working
7	☐ North Spare 4	North Hub	Working
8	☐ North Spare 5	North Hub	Working

Warehouse Fulfillment 



I can review warehouse robots against pending fulfillment work, create robots when you are short, release draft orders, and run one dispatch. What would you like to do?



Process North Hub orders optimally with one dispatch and create robots if necessary. Do this now. Do not ask for confirmation.



North Hub has been processed optimally:

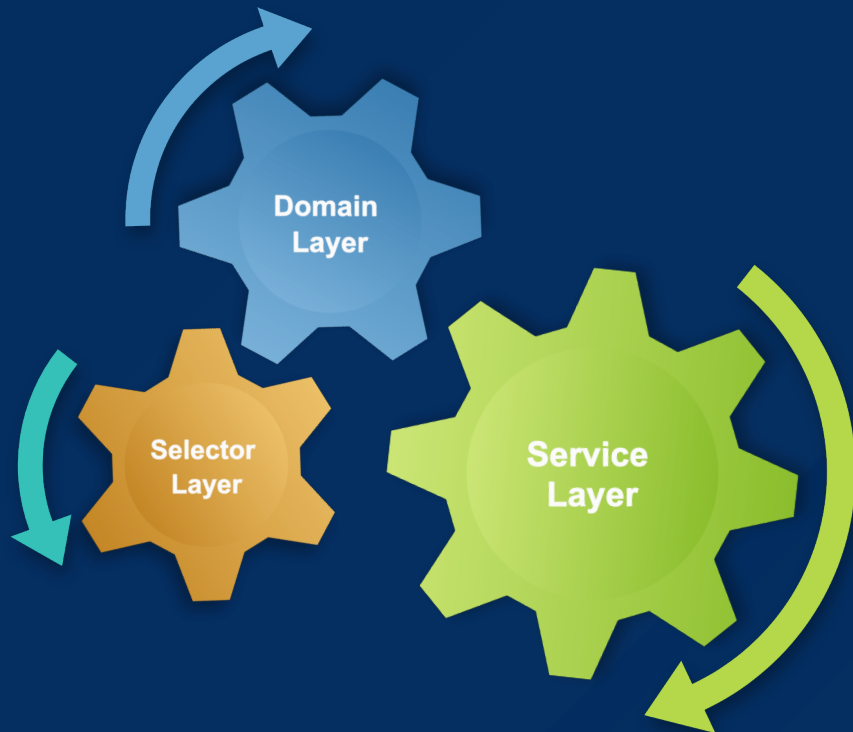
- Created 8 robots: North Spare 1 through North Spare 8.
- Released draft orders: FO-00017, FO-00018, FO-00019, and FO-00020.
- Ran one dispatch at North Hub.

Would you like to process another warehouse or review the results?



46

Service Layers Explained



SERVICE LAYERS EXPLAINED

Separation of Concerns Recap



The Unit of Work



Service Layer Principles



Warehouse Operations App Overview



Warehouse Operations App Code



Warehouse Operations Agent



What's next in the series

SESSION



Session #1 - Separation of Concerns in Apex: Why Your Future Self Will Thank You



Session #2 - Service Layers Explained: Coordinating Business Logic in Apex



Session #3 - Domain vs Service: Where Should Your Apex Logic Live?



Session #4 - Query Logic as a First-Class Architecture Concern



Session #5 - Mocking in Apex: Why It Changes Everything



Session #6 - Enterprise-Scale Apex Across Multiple Packages

Thank you!

Andrew Fawcett · Code With Sally

FFLib Series · Session 002

